Jolt-QED: Formally Verifying Bytecode Expansions In Lean

Ari Biswas[1], Quang Dao[2], Daniel Ross[3], and Justin Thaler[4]

[1] University of Warwick & Layer Zero Labs
tcs@randomwalks.xyz
[2] Carnegie Mellon University & Layer Zero Labs
qvd@andrew.cmu.edu
[3] Independent Researcher
daniel_144@toastertime.net
[4] a16z crypto & Georgetown University
justin.r.thaler@gmail.com

Abstract. In this work, we verify, using the Lean 4 proof assistant, that the Jolt zk-VM's expanded programs faithfully emulate a RISC-V CPU. Prior work translated the official Sail specification of RISC-V into Lean. We extend this model to define the Jolt ISA in Lean. We build a generator that runs Jolt's Rust bytecode expander and renders its output as Lean programs. Finally, for 51 out of 58 such programs, we prove these expansions are equivalent to their corresponding RISC-V instructions. This lays the groundwork for proving completeness and soundness of further stages of Jolt's proving pipeline.

1 Introduction

Jolt [2] is a state-of-the-art zero-knowledge virtual machine (zk-VM) that powers the proving machinery of blockchains in research and industry [9]. A traditional virtual machine executes a guest program compiled for a specified instruction set architecture (ISA), such as RISC-V, ARM or x86, by emulating the guest CPU in software. A zk-VM is a virtual machine with the added responsibility that it is required to output an efficiently[1] checkable cryptographic proof[2] that it executed every instruction of the guest program correctly. Thus, Jolt, being a zk-VM, can be logically split into the tracer and the prover. The tracer takes a RISC-V guest program, denoted here with x, executes instructions in order, and generates a witness w. In the language of computational complexity theory, the guest program x is often referred to as the problem instance, and the honest trace witness w is referred to as the NP witness which can be compared with the program to validate that the output was generated by executing the program's actual instructions.

[1] By efficient we mean that, if T is the time taken to run the program, verification should take time sub-linear in T.

[2] These proofs are often referred to as Succinct Arguments of Knowledge (SNARK) in the literature. The *zero knowledge* in the zk-VM is misleading and is an artefact of poor nomenclature originating in the early days of SNARK populism. The defining property of the proof is that it is efficiently verifiable. The proof may or may not be zero knowledge.

Generating an NP witness on its own accomplishes little. A client wishing to confirm the result came from the program could simply run the program to generate its own witness w', and compare it against the prover's witness w. The challenge is to generate a substantially cheaper procedure for validating program execution. The Jolt prover's task, then, can be viewed as compressing the witness w to output a proof $\pi := \pi(w, x)$ that a client can efficiently process to validate whether the program was in fact run correctly.

To efficiently generate a proof, the Jolt zk-VM makes use of instruction tables whose *multilinear extensions* can be evaluated efficiently [2]. Jolt's implementation of the Shout lookup prover [15] also requires their multilinear extensions to have *prefix-suffix structure* [10]. Intuitively, we view the bits of an instruction's inputs as variables and represent its output as a field element. Its multilinear extension is the unique polynomial that agrees with the instruction on Boolean inputs and has degree at most one in each variable. The `AND` instruction, for example, has a particularly simple multilinear extension. Let $m \in \mathbb{N}$. For any $\vec{x}, \vec{y} \in \{0,1\}^m$, let $f_{\text{AND}}\left(\vec{x}, \vec{y}\right) = (x_{m-1} \| \ldots \| x_0)$ & $(y_{m-1} \| \ldots \| y_0)$ be the bitwise AND operation. We interpret its output in a prime field $\mathbb{F}$ of characteristic greater than 2^m. Its multilinear extension, for any $\vec{x}, \vec{y} \in \mathbb{F}^m$, is:

$$\widetilde{f}_{\text{AND}}\left(\vec{x}, \vec{y}\right) = \sum_{i=0}^{m-1} 2^i \cdot x_i \cdot y_i. \tag{1}$$

On Boolean inputs, $x_i y_i$ is the AND of the two bits, so this polynomial gives the required output. Each variable occurs with degree at most one. We can evaluate the same expression at arbitrary $\vec{x}, \vec{y} \in \mathbb{F}^m$ using $O(m)$ field operations, even though the instruction's full table has 2^{2m} entries. This efficient evaluation is one of the two properties that Jolt needs.

`AND`'s MLE also satisfies the prefix-suffix structure. Order the variables as $(x_{m-1}, y_{m-1}, \ldots, x_0, y_0)$. For any integer $0 < k < m$, split this sequence between the pairs (x_k, y_k) and (x_{k-1}, y_{k-1}). We can write

$$\widetilde{f}_{\text{AND}}\left(\vec{x}, \vec{y}\right) = \underbrace{\sum_{i=k}^{m-1} 2^i \cdot x_i \cdot y_i}_{P} \cdot 1 + 1 \cdot \underbrace{\sum_{i=0}^{k-1} 2^i \cdot x_i \cdot y_i}_{S}. \tag{2}$$

Here, P depends only on the prefix variables and S only on the suffix variables. This is a sum of two products of multilinear polynomials on separate sets of variables, giving the required prefix-suffix structure. Thus, the `AND` table's multilinear extension has both properties.

For instructions such as `SLL`, the difficulty is obtaining prefix-suffix structure suitable for Jolt's lookup prover. The approach the Jolt zk-VM takes is to re-write the guest RISC-V program into a program in the Jolt ISA. Instructions in this ISA are supported by arithmetic checks and lookup tables whose multilinear extensions can be evaluated efficiently and have the required prefix-suffix structure. Thus, Jolt replaces RISC-V instructions that it does not support directly with a sequence of one or more Jolt instructions. In order to aid this emulation process, the Jolt CPU includes the usual RISC-V registers plus some additional "virtual" general-purpose registers in which it stores intermediate results.

For example, the RV64 `SLL rd, rs1, rs2` instruction computes $f(x, y) = x \ll (y \bmod 64)$, where $x, y \in \{0,1\}^{64}$ represent the contents of registers `rs1` and `rs2` respectively. $\ll$ denotes the logical left bit shift operator, and the value $f(x, y) \in \{0,1\}^{64}$ is written into the contents of `rd`. For `rd` different from `x0`, Jolt replaces this instruction with the following sequence.

```
VirtualPow2  v40, rs2
MUL          rd,  rs1, v40
```

Here, `VirtualPow2 v40 rs2` sets the scratch register `v40` to $2^{y \bmod 64}$, where y is the contents of `rs2`. As `VirtualPow2` is not present in the RISC-V ISA, it is referred to as a virtual instruction. `MUL`, on the other hand, is the extension of the RISC-V instruction `MUL` to the Jolt ISA by allowing it to operate on both RISC-V general-purpose registers and Jolt ISA virtual registers. Multiplication modulo 2^{64} by this power of two gives the same value in `rd` as RV64 `SLL`. When `rd = x0`, the expansion instead emits `ADDI x0, x0, 0`, since the result is discarded. This separates the computation of the power of two from the multiplication, expressing the shift using operations supported by Jolt's proving machinery. In summary, the Jolt ISA can be viewed as a strict subset of the RISC-V ISA, plus some virtual instructions, that can operate on RISC-V registers, plus some virtual registers.

Reflecting back on the job of the tracer, in practice, witness generation proceeds in two steps. The Jolt tracer first translates all RISC-V instructions into one or more Jolt ISA instructions (the Jolt documentation refers to this process as *bytecode expansion*). Once translation is complete, the Jolt CPU executes Jolt instructions to generate the witness. *Thus, the Jolt proof guarantees that it ran every Jolt (not RISC-V) instruction of the transformed guest program correctly.* With the execution expressed using these Jolt instructions, the prover can use the arithmetic checks and efficient multilinear evaluations to compress the witness into an efficiently checkable proof. This added bytecode expansion step, however, creates another potential for failure. RISC-V execution semantics are complex, and it is now necessary to ensure that the expanded Jolt ISA versions of all RISC-V instructions implement equivalent semantics.

Our primary contribution in this work is to use the Lean 4 proof assistant to prove that each covered Jolt bytecode expansion faithfully implements its corresponding RISC-V instruction. In other words, under the stated assumptions and starting from corresponding initial machine states, executing the expansion yields the same final RISC-V CPU state as executing the original instruction.

An incorrect expansion can compromise soundness by permitting an incorrect RISC-V result, or completeness by rejecting a valid RISC-V execution. The `DIVW` bug described in Section 6.1 is an example of the latter. Now while the importance of zk-VM correctness is easily justified, two immediate questions arise.

First, why apply formal verification methods for verifying Jolt's bytecode expansion phase? One might ask whether fuzzing or differential testing could provide comparable assurance with far less effort. Randomised testing has uncovered hundreds of compiler bugs [22], and recent work by Christoph Hochrainer, Valentin Wuestholz, and Maria Christakis [8] found eleven soundness and completeness bugs in three of six tested zk-VMs. Although fuzzing and differential testing are valuable tools for finding concrete failures,

they do not guarantee the absence of bugs. Lennart Beringer, Adam Petcher, Katherine Q. Ye, and Andrew W. Appel [3] argue that testing and fuzzing are insufficient to establish functional correctness and cryptographic security, and use the Coq proof assistant to verify an OpenSSL HMAC implementation against its FIPS functional specification. Furthermore, Jolt's unit tests already use differential testing as a security check. Despite these tests, the incompleteness error described in Section 6 went undetected until it was found through formal verification in this work. Another motivating factor is that Ethereum Foundation [5] identifies formal verification as part of the security foundation for L1 zkEVMs and invests in the verification of critical components, security proofs, and specifications that match deployed code. Bugs have been identified in SNARK circuits [21], further motivating the use of formal verification to establish their correctness and security.

The second question that may arise is why we use the Lean 4 proof assistant. One motivating factor is that researchers from the University of Cambridge, Galois, University of Edinburgh, and Lindy Labs have developed a Sail-to-Lean transpiler that generates Lean definitions from the official RISC-V Sail model [14] [6]. Sail is an industry-accepted, domain-specific language for describing ISAs. The resulting Lean definitions type-check and provide a *trusted* representation of the RISC-V specification against which we can state and prove the equivalence of each Jolt expansion and its corresponding RISC-V instruction. This generated Lean specification has already been used as a reference in formal-verification efforts for other zkVMs. Succinct and Nethermind verified the core RV64 chips of SP1 Hypercube against the official Sail RISC-V specification [16]. Earlier, Nethermind used Lean to formalise selected components of the RISC Zero zkVM [12] and to prove that extracted constraints for SP1's `AddSub` chip conform to its specification [11]. The Ethereum Foundation has funded Lean-based verification work beyond the RISC-V model, including projects on the Jolt zkVM, SP1 ALU chips, and the ArkLib SNARK library [4]. ArkLib is being developed as a modular Lean framework for formalising SNARKs in general, with current work targeting completeness and round-by-round knowledge soundness for selected protocols, including sum-check, Spartan, FRI, STIR, WHIR, and Binius [20]. VCVio provides a separate Lean framework for formalising cryptographic security arguments, including a formally verified forking lemma and Fiat–Shamir transform [19]. Mathlib provides a general-purpose library of formalised mathematics for Lean [18]. As a consequence of these projects, we believe Lean 4 is a practical environment for formalising the algebraic and cryptographic components used by Jolt.

The remainder of the document is organised as follows. The phrase "formal verification" has become quite overloaded in recent years. Given the promise of strong guarantees of proof assistants, it is tempting to use it as a broad umbrella to advertise the complete security of a zk-VM. Jolt is written in Rust, while our formal verification results are stated and proved in Lean. Relating the Rust implementation to the Lean theorems requires explicit assumptions about modelling, extraction and compilation. In Section 2, we explain how the Lean theorems relate to the implementation and which components of the verification pipeline are trusted. In Section 3, we describe the Lean representation of the RISC-V CPU, including its machine state and execution semantics, before defining the Jolt ISA on top of it. Theorems are only true if the underlying assumptions apply. In

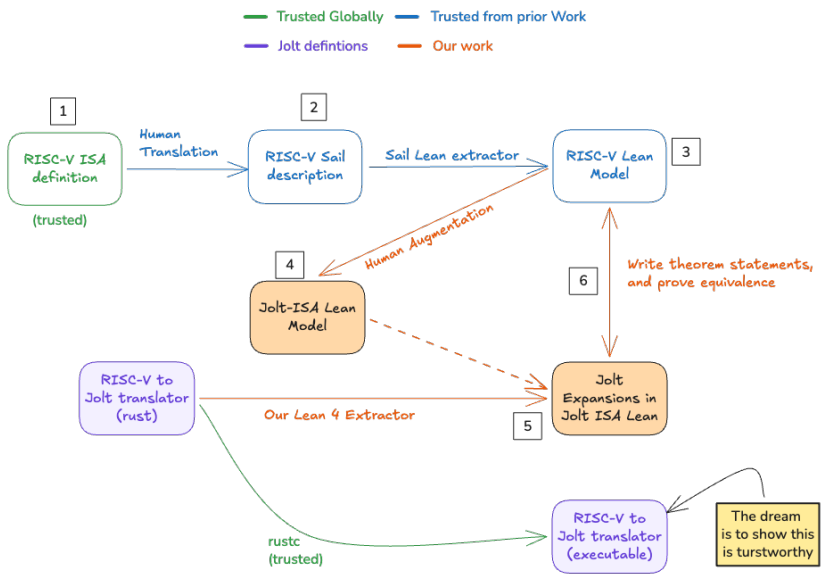


Fig. 1. Items in green are trusted components, including the `rustc` compiler and the RISC-V ISA specification. Items in blue are prior work on which this verification relies, including the extraction of RISC-V semantics from Sail to Lean. Items in purple describe the Jolt source code and executable. Items in orange describe the Lean source used in this work to verify Jolt's bytecode expansions.

Section 4, we make explicit the assumptions with references to the Jolt codebase under which our equivalence theorems hold. These assumptions are just as important as the final theorem statement, and their presence *must* be justified. In Section 5, we describe how we obtain Lean expansions of RISC-V instructions from their Rust definitions. Finally, in Section 6, we summarise the bugs and architectural mis-specifications uncovered during our work and discuss the limitations of the guarantees established by formal verification.

2 The Process of Verification

Formal verification of software is intended to create improved trust in code that one wishes to rely upon. Before we describe how we formally verified equivalence of the specific software artefacts under study, we will seek to describe what those software artefacts are, how they relate to the software artefacts that are intended for widespread use (and therefore require improved trust), and how our verification project generates trust in those artefacts. In particular, this process distinguishes what has been proved from the components whose correctness is trusted, such as the Rust compiler, and then identifies areas for future verification.

Figure 1 summarises the process of trust propagation. It outlines what we verify and the dependencies we trust when relating the bytecode expander's executable behaviour

to RISC-V instructions. Below we describe the process of trust propagation with item numbers acting as references back to the figure.

1. The starting point is the RISC-V ISA architectural specification defined by the chip manufacturers [13].
2. Prior work [14] translates the natural-language RISC-V specification into Sail [1], a language for defining processor instruction-set semantics. This Sail model has been adopted by RISC-V International.
3. Researchers at the University of Cambridge, Galois and LindyLabs, with contributions from the University of Edinburgh, developed the Sail-to-Lean translation used to generate a Lean 4 model of RISC-V [6]. We prove equivalence against this generated model. The translated code is not executable; however, it is still suitable for theorem proving. Our guarantees depend on this Lean model faithfully representing the RISC-V specification.
4. We extend the Sail-derived RISC-V model to define the Jolt state and instruction semantics in Lean (see Section 3). The instruction semantics are hand-transcribed from the tracer's Rust execution methods, including those for virtual instructions. Agreement between these Rust methods and their Lean definitions is a trusted assumption when applying the results to Jolt. This manual correspondence is a dependency of the Lean Jolt ISA model in Figure 1.
5. We obtain Lean expansion programs by running Jolt's production Rust expander on concrete source instructions and printing parameterised templates over the Lean ISA from Step 4. The generator is described in Section 5. Applying the Lean results to Rust assumes that these templates faithfully represent the expander's output for all covered operands.

6. Except for 7 instructions discussed in Section 6, we prove agreement between the Sail-derived RISC-V semantics and the corresponding Lean Jolt semantics under instruction-specific assumptions. For expansions that use untrusted advice, we prove both completeness and soundness. Completeness says if the prover were to use the correct advice, then the expanded program would be equivalent to the corresponding RISC-V program. Soundness says that any other advice would cause the Jolt program to panic.

2.1 How does this generate trust in the bytecode expander binary code?

Figure 1 makes it abundantly clear that there is some distance between the executable binary code we would like to trust and the Lean 4 models for which we are verifying equivalence. In more detail, first, we remark that verifying emulators according to ISA specifications is a fairly standard business involving test vectors that are, largely, ISA-driven and thus can be centralised. For our purposes, the Sail Lean extractor is being treated as trusted, and we leave any validation of that to future work. This allows us to trust (subject to the caveats concomitant to the above discussion) that the RISC-V Lean 4 model faithfully represents the RISC-V ISA. The Jolt ISA model extends the RISC-V model with explicit definitions for the additional state and instruction semantics. These definitions must be reviewed against the corresponding Rust execution methods.

As these definitions are translations of small blocks of straight-line code, it is reasonable to manually verify them.

For automatically extracted expansions, the generator reuses the production Rust expander. Its parameter recovery and rendering rules are additional trusted code; Section 5 explains their role and the checks they perform. Relating the Rust source to the executable additionally relies on compiler correctness. For instructions which are native to both RISC-V and Jolt, we still prove equivalence. The native-instruction proofs establish agreement between our Lean instruction definitions and Sail under the stated hypotheses. Although this does not officially guarantee that our translations are correct, it gives us a layer of confidence that they are likely correct. The expansion proofs establish the combined behaviour of the Lean instructions used in each expansion. Neither result absolutely guarantees that the hand-transcribed definitions agree with the Rust execution methods. In particular, a transcription error in a virtual instruction can leave an expansion proof valid in Lean while its Rust execution behaves differently, but we argue the chances of this happening accidentally are low. Still we make a point to mention this explicitly as all our theorems rest on the assumption that we did the translation and expansion correctly.

3 Modelling Stateful Computation In Lean

As discussed in Section 1, one of the motivating factors behind formalising Jolt in Lean is that we already have a RISC-V specification written in Lean. Given that we are going to demonstrate techniques for proving things about program execution in Lean, it will help to explain in some detail what modelling a RISC-V CPU in Lean looks like.

3.1 The Sail RISC-V Lean Model

As the Jolt ISA builds on top of a RISC-V ISA, our starting point will be to understand the RISC-V Lean model generated by the Sail compiler as described in Section 2. While the RISC-V Sail specification is quite full-featured, modelling elements such as the TLB cache and multiple harts, we will limit our explanation to a subset of types/structure elements relevant for our concerns: registers, memory, and CPU error states. As such, our description will faithfully represent that subset as reified in the Sail Lean model, but will not be exhaustive. We remark that every code snippet described in this subsection is part of the trusted transpilation. We did not write the Lean code below. We describe it in order to better understand how we extend it, when we do write Lean code for the Jolt ISA.

RISC-V Machine State in Lean The RISC-V Lean model represents its machine state as `SequentialState`, a structure parameterised by a dependent register-type function `RegisterType : Register → Type`. (Note that by using `Sequential State` we assume that instructions are executed in order, and we do not model concurrent instruction execution. For the purpose of proving equivalence to Jolt bytecode expansions, this distinction does not matter.)

```
structure SequentialState (RegisterType : Register → Type) where
  regs : Std.ExtDHashMap Register RegisterType
  mem : Std.ExtHashMap Nat (BitVec 8)
  sailOutput : Array String
  -- ... other elements such as current CPU states, etc.
```

Thus in this model, memory is modelled as byte-addressable (i.e., the values are `Bit Vec 8`), but the addresses—the keys of the hash map—are modelled as of type `Nat`.

Registers are modelled using Lean's built-in dependent hash map type, which accounts for the fact that different sorts of registers may have different value types (e.g. some registers might be 32-bit whereas others might be 64-bit). Thus the type signature of `Std.ExtDHashMap` receives the key type (`Register`), which will be an inductive type whose terms simply enumerate every RISC-V register, and not the value type, but a function which specifies, for each possible key type, what the type of the corresponding value will be:

```
inductive Register : Type where
  | PC
  | x1 | x2 | x3 | ... | x31   -- general-purpose registers
  -- ... CSRs, vector registers, debug state, ~170 variants total ...

abbrev RegisterType : Register → Type
  | .PC => (BitVec 64)
  | .x1 => (BitVec 64)   -- all x1-x31 map to BitVec 64
  | .x2 => (BitVec 64)
  -- ... other register types such as hart state, TLB entry, etc.
```

After instantiating `SequentialState` with the `RegisterType` function defined above, we write the resulting RISC-V machine state as `SailState`.

```
abbrev SailState := SequentialState RegisterType
```

Program Execution in Lean Modelling CPU instruction execution is an inherently stateful process. As Lean is a pure functional language, the Sail model encodes the stateful nature of CPU execution using Lean's built-in error-state monad, `EStateM`. The relevant parameters for the EStateM monad are a state type and an error type (and an answer type, which we will not make use of). The state we will use is the `SailState` introduced above, and the error type will be an inductive type that enumerates the various error states a CPU can enter (e.g. `Exit` for when the code requests a halt, or `OutOf MemoryRange` for when code executes outside the valid memory range, or a further `User` type for specific processor implementations.[3]):

[3] The `Error` type definition comes from the generic Lean library that is used across all Lean processor models generated by Sail, and thus leaves the processor-specific exception types to be specified on an ISA-by-ISA basis.

```
inductive Error (ue : Type) where
  | Exit
  | OutOfMemoryRange (n : Nat)
  | User (e : ue)
  -- .. other error conditions
```

An instruction computation has type `EStateM (Error exception) SailState ExecutionResult`: it takes an initial `SailState` and returns either `.ok result s'`, containing an `ExecutionResult` and the updated state, or `.error err s'`, containing an `Error exception` and the state at the point of failure. Both constructors retain the state because a computation may modify it before returning an error. An `.ok` result means that the monadic computation returned an `ExecutionResult`; that value can represent normal retirement, a trap or another architectural outcome. Most instructions retire normally with `Retire_Success`. The remaining variants cover traps, memory exceptions, and privilege changes, most of which we do not have to deal with when arguing equivalence.

```
inductive ExecutionResult where
  | Retire_Success (_ : Unit)
  | ExecuteAs (_ : instruction)
  | Enter_Wait (_ : WaitReason)
  | Illegal_Instruction (_ : Unit)
  | Virtual_Instruction (_ : Unit)
  | Trap (_ : (Privilege × ctl_result × xlenbits))
  | Memory_Exception (_ : (virtaddr × ExceptionType))
  | Ext_CSR_Check_Failure (_ : Unit)
  | Ext_ControlAddr_Check_Failure (_ : ext_control_addr_error)
  | Ext_DataAddr_Check_Failure (_ : ext_data_addr_error)
  | Ext_XRET_Priv_Failure (_ : Unit)
```

Together, these instantiate `SailM` — the monad in which all transpiled RISC-V instructions are written:

```
abbrev SailM := EStateM (Error exception) SailState
```

A computation of type `SailM` α is a function from an initial state to an error-state result containing a return value of type α or an error. Shown below is an example of how the trusted translation defines how the `ADDIW` instruction updates the RISC-V state.

```
def execute_ADDIW (imm : (BitVec 12)) (rs1 : regidx) (rd : regidx) :
  SailM ExecutionResult := do
  let result ← do (pure ((← (rX_bits rs1)) + (sign_extend (m := 64) imm
  )))
  (wX_bits rd (sign_extend (m := 64) (Sail.BitVec.extractLsb result 31 0)
  ))
  (pure RETIRE_SUCCESS)
```

3.2 Modelling Jolt State

Having explained the Lean model of a RISC-V CPU, we now proceed to explain our extension of this to a Lean model of a Jolt ISA CPU. All code[4] snippets described in this section are *new*. They were *not* part of the trusted Sail transpilation.

The Jolt ISA The Jolt ISA extends a subset of the RISC-V ISA with virtual instructions and registers. Its register address space has 128 slots: 32 architectural registers and 96 virtual registers. Our Lean model stores architectural registers in the Sail state and uses a separate seven-bit-indexed function for virtual registers. The instructions within the Jolt ISA consist of the subset of RISC-V instructions that are decomposable, along with a further set of "Virtual" instructions that are also MLE evaluation friendly, and whose purpose is to render the remaining RISC-V instructions implementable as relatively straightforward compositions of Jolt ISA instructions. The `Src` and `Dst` operand types let Jolt instructions use both virtual and RISC-V general-purpose registers.

The Jolt CPU model in Lean To implement this in Lean, we define the Jolt CPU State as a structure that contains the `SailState` from the RISC-V CPU model above:

```
structure SailJoltState where
  sail  : SailState
  vregs : BitVec 7 → BitVec 64 := fun _ => 0
```

The `sail` field holds the full RISC-V architectural state. The `vregs` field maps each 7-bit index to a 64-bit word with the default value initialised to 0[5]. Unlike `sail.regs`, which is a dependent hashmap, `vregs` is a plain function and every virtual register has type `BitVec 64`. This implies that we do not need to assume that the register is readable. Unlike a hash-map which might have missing keys, a function can be evaluated successfully for every term in its domain (see Section 4 for more details). Similar to `SailM`, Jolt computations live in `JoltMonad`, which is the `EStateM` monad with `Sail JoltState` as the type that describes machine state.

```
abbrev JoltMonad (α : Type) := EStateM (Error exception) SailJoltState α
```

To define the Jolt instructions in Lean, we use an inductive type `JoltISA.Instr`. Its constructors fall into two groups:

1. *RISC-V-like instructions*, which correspond directly to RISC-V opcodes.
2. *Virtual instructions* (prefixed `Virtual`), which have no RISC-V counterpart and exist only in the Jolt model.

[4] The full source code can be found at `https://github.com/LayerZero-Labs/jolt-qed/`

[5] In the Rust codebase, they use virtual registers 0-31 to model RISC-V general-purpose registers. However, in Lean, the general-purpose registers are already defined to be inductive types that key into the hash map. Thus, to stay faithful to the Sail model and be able to prove theorems, we model RISC-V registers using the same inductive types and never use virtual registers 0-31 to prove theorems.

```
inductive Instr where
  -- RISC-V-like instructions
  | ADDI  (dst : Dst) (src : Src) (imm : BitVec 12)
  | ANDI  (dst : Dst) (src : Src) (imm : BitVec 12)
  | ADD   (dst : Dst) (lhs rhs : Src)
  | SUB   (dst : Dst) (lhs rhs : Src)
  | BEQ   (lhs rhs : Src) (imm : BitVec 13)
  | LD    (faultClass : LoadFaultClass) (dst : Dst)
          (base : Src) (imm : BitVec 12)
  | SD    (base value : Src) (imm : BitVec 12)
  -- ...
  -- Virtual instructions
  | VirtualSignExtendWord    (dst : Dst) (src : Src)
  | VirtualAssertHalfwordAlignment
      (base : regidx) (imm : BitVec 12) (fault : ExceptionType)
  | VirtualAssertWordAlignment
      (base : regidx) (imm : BitVec 12) (fault : ExceptionType)
  | VirtualAdvice            (dst : Dst) (value : BitVec 64)
  | VirtualAssertEQ          (lhs rhs : Src) (imm : BitVec 13)
  | VirtualNegateIf          (dst : Dst) (signSource value : Src)
  -- ...
```

For `LD`, `LoadFaultClass` selects ordinary-load (`.normal`) or atomic (`.amo`) fault reporting. As the Jolt CPU can read from and write to both RISC-V general-purpose registers and virtual registers, we define generic inductive types called `Src` and `Dst` to model source and destination registers.

```
inductive Src where
| vreg : VReg → Src
| xreg : regidx → Src

inductive Dst where
| vreg : VReg → Dst
| xreg : regidx → Dst

def readSrc : Src → JoltMonad (BitVec 64)
  | .vreg vr => readVReg vr          -- direct function application,
  always .ok
  | .xreg rs => liftSail (rX_bits rs) -- hashmap lookup, requires read
  succeeds assumption

def writeDst : Dst → BitVec 64 → JoltMonad Unit
  | .vreg vr, v => writeVReg vr v    -- rejects indices below 32
  | .xreg rd, v => liftSail (wX_bits rd v) -- always .ok
```

The virtual-register write operation rejects indices 0–31, which are reserved for architectural registers and must be accessed through `.xreg`. Reads from the virtual-register function are total.

We also write helper methods `readSrc` and `writeDst` to read from and write to virtual or general-purpose RISC-V registers. The `liftSail` function runs a Sail

computation on the `sail` component of `SailJoltState`, retaining the rest of the Jolt state.

Above we describe the instruction signatures. We still have to define how these instructions affect machine state, i.e. the state transition function for each of these instructions. Instruction semantics are given by `execInstr`, which maps each `Instr` to a `JoltMonad ExecutionResult`.

```
-- jolt-qed/JoltBytecode/JoltISA/Semantics.lean
def execInstr : Instr → JoltMonad ExecutionResult
  | .ADDI dst src imm => do
      let x ← readSrc src
      writeDst dst (x + sign_extend (m := 64) imm)
      pure RETIRE_SUCCESS
  | .ADD dst lhs rhs => do
      let x ← readSrc lhs
      let y ← readSrc rhs
      writeDst dst (x + y)
      pure RETIRE_SUCCESS
  -- ... one case per constructor
```

The logic for the instruction semantics is specified in the `tracer` crate of Jolt in the `tracer/src/instruction/<instr-name>.rs` files. Each file contains an `exec` method that describes how the instruction updates the Jolt CPU.

```
impl ADDI {
    fn exec(&self, cpu: &mut Cpu, _: &mut <ADDI as RISCVInstruction>::
  RAMAccess) {
        cpu.write_register(
            self.operands.rd as usize,
            cpu.sign_extend(
                cpu.x[self.operands.rs1 as usize]
                    .wrapping_add(normalize_imm(self.operands.imm, &cpu.
  xlen)),
            ),
        );
    }
}
```

Remark. The Lean execution semantics for Jolt instructions are hand-transcribed. That is, we have no automated procedure to go from the Rust implementation above to the following Lean implementation. In Section 2, we mentioned that to describe the Jolt ISA in Lean, we needed a little bit of human-augmentation. This includes the virtual instructions, which have no direct Sail counterpart. As CPU semantics are given by short, simple blocks of code, we do not envisage this translation introducing significant risk to the final guarantees. Additionally, the proof of equivalence against the trusted Sail description of the same instructions gives us confidence that at least the native RISC-V instructions have been implemented correctly. Still, it is imperative we mention this clearly off the bat, to ensure we are fully transparent about what is the final artefact we are verifying. *Any* issues in the translation process could jeopardise the guarantees of our theorems.

Now we describe the bytecode expansion process. A supported RISC-V instruction without a direct `JoltISA.Instr` constructor is represented by an expansion of type `Program`:

```
inductive Program where
  | done  (result : ExecutionResult)
  | instr (instr : Instr) (next : Program)
```

A program is recursively defined to be either `done` or a Jolt instruction followed by another Jolt program. As a concrete example, the RISC-V `SLL rd, rs1, rs2` instruction (shift-left-logical) has no direct `Instr` constructor. Its Jolt expansion is `sll ProgramAuto`, which uses two instructions when `rd` is nonzero and a no-op when `rd = x0`:

```
def sllProgramAuto (rd rs1 rs2 : regidx) : Program :=
  if isX0 rd then
    .instr (.ADDI
      (.xreg (regidx.Regidx 0)) (.xreg (regidx.Regidx 0))
      (0 : BitVec 12)) <|
    .done RETIRE_SUCCESS
  else
    .instr (.VirtualPow2
      (.vreg (BitVec.ofNat 7 40)) (.xreg rs2)) <|
    .instr (.MUL
      (.xreg rd) (.xreg rs1) (.vreg (BitVec.ofNat 7 40))) <|
    .done RETIRE_SUCCESS
```

`VirtualPow2` computes 2^s, where s is the value of the low six bits of `rs2`; `VirtualPow2W` uses five bits and is used for `SLLW`. The Rust code below constructs the two-instruction branch; the surrounding expansion pipeline handles `rd = x0`.[6]

```
asm.expand_i(
    SourceInstructionKind::VirtualPow2,
    v_pow2.operand(),
    reg(rs2(instruction)?),
    0,
);
asm.emit_r(
    Kind::MUL,
    reg(rd(instruction)?),
    reg(rs1(instruction)?),
    v_pow2.operand(),
);
```

Expansions such as `SLL` are extracted by running the Rust expander and rendering its output as parameterised Lean programs. See Section 5 for the extraction procedure and its trust assumptions.

A `Program` is a sequence of Jolt instructions. `execProgram` recursively describes how this sequence changes the Jolt CPU state. For each instruction, we invoke the `exec`

[6] https://github.com/abiswas3/jolt/blob/3bde0cbd1a5ec5d7b9da72396d5076a0dd72b37f/crates/jolt-program/src/expand/shifts/sll.rs#L14-L25

`Instr` method to run the instruction. If an instruction retires with `RETIRE_SUCCESS`, the interpreter continues with the remaining instructions. Otherwise, the result is returned immediately and the remaining instructions are skipped. Monadic errors also stop execution.

```
def execProgram : Program → JoltMonad ExecutionResult
  | .done result    => pure result
  | .instr instr rest => do
      match ← execInstr instr with
      | .Retire_Success () => execProgram rest
      | result             => pure result  -- short-circuit on any non-
  retire outcome
```

Relating the Jolt and RISC-V CPU models To state equivalence between a Jolt computation and a RISC-V computation, we need to project a Jolt Monad result down to a `SailM` result. That is, after running a sequence of state transitions of the Jolt CPU, we should be able to ask what the RISC-V CPU looks like now. The method that allows us to do this is the `systemProjectResult` shown below.

```
def systemProjectResult
    (r : EStateM.Result (Error exception) SailJoltState α) :
    EStateM.Result (Error exception) SailState α :=
  match r with
  | .ok a js' => .ok a (systemProject js')
  | .error e js' => .error e (systemProject js')
```

In simple English, it says if you give me the result of a Jolt computation (successful or not), then we return a result of a RISC-V computation which we know has type `EState M.Result (Error exception) SailState` α (the output type). The body of the method says that this is achieved by taking the Jolt computation's updated Jolt state and projecting it down to a RISC-V state with the `systemProject` method described below.

```
def systemProject (js : SailJoltState) : SailState :=
  { js.sail with
    regs :=
      ((((((js.sail.regs
        |>.insert Register.mtvec (js.vregs JoltISA.trapHandlerVReg))
        |>.insert Register.mscratch (js.vregs JoltISA.mscratchVReg))
        |>.insert Register.mepc (js.vregs JoltISA.mepcVReg))
        |>.insert Register.mcause (js.vregs JoltISA.mcauseVReg))
        |>.insert Register.mtval (js.vregs JoltISA.mtvalVReg))
        |>.insert Register.mstatus (js.vregs JoltISA.mstatusVReg)) }
```

At first glance, the above code block appears unnecessarily complicated. Intuitively, the projection should just return the RISC-V part of the Jolt state, i.e., we should just output `js.sail`. This is exactly what we do, but before we output the RISC-V state, we write the values in 6 virtual registers to 6 specific locations in the register file hashmap. Jolt's six modelled CSRs occupy virtual-register indices 34–39; indices 32 and 33 hold reservation state. The trusted RISC-V state register file hashmap has keys that model

these CSRs explicitly. Thus, if a Jolt program ever writes to these CSRs via virtual register writes, this write should be reflected in the RISC-V state. Without this prelude, a Jolt program can write to virtual registers modelling CSRs while the RISC-V program does not, and a theorem can still pass. So far, we have defined Jolt state and how to project down from Jolt to RISC-V state.

Now that we have given the definition of a RISC-V CPU and a Jolt CPU in Lean, as well as a way to take a Jolt CPU state and determine the corresponding RISC-V CPU state, we are now ready to describe the main set of theorems that we prove here. This will be the work of the next section.

4 Theorem Statements And Assumptions

For the covered expansions listed in Section 6, we prove correctness under an instruction-specific assumption bundle. For expansions without advice, the statement equates the projected Jolt execution result with the corresponding Sail execution result from related initial states. Advice-using expansions have separate completeness and soundness statements. These instruction-level guarantees compose by induction over finite executions of covered instructions: the initial states must correspond, each instruction's theorem premises must hold when it executes, and each normally retiring output state must supply the next input at matching architectural boundaries. Any intervening state updates must preserve the correspondence. This composition argument assumes the premises at every step; it does not require deriving them from the initial state alone. The theorems formalised here establish the individual-instruction steps. The details are best illustrated by the theorem for the SLL expansion in Section 1.

```
def sllProgramEqSailStatement
    -- Arguments for the SLL instruction
    (rs2 : regidx)
    (rs1 : regidx)
    (rd : regidx)
    -- Initial Jolt State
    (js : SailJoltState)
    -- Assumption Bundle
    (_h : BinarySourceReadWithLinkedCSRs rs2 rs1 js) : Prop :=
    -- Theorem statement
  System.systemProjectResult
      ((JoltISA.execProgram (JoltISA.sllProgramAuto rd rs1 rs2)).run js)
  =
    (execute_RTYPE rs2 rs1 rd rop.SLL).run js.sail
```

The input arguments are direct: `js` is the Jolt state before executing the Jolt program, and `js.sail` is the corresponding RISC-V state before executing SLL. The theorem statement is:

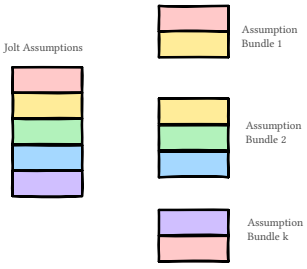


Fig. 2. The primitive assumptions are defined in `Assumptions.lean`; `Bundles.lean` groups them for each instruction family. Reviewing the primitive definitions together with the bundle fields makes the hypotheses of each theorem explicit.

```
System.systemProjectResult
      ((JoltISA.execProgram (JoltISA.sllProgramAuto rd rs1 rs2)).run js)
  =
    (execute_RTYPE rs2 rs1 rd rop.SLL).run js.sail
```

We now break down the theorem statement.

`(execute_RTYPE rs2 rs1 rd rop.SLL).run` is the RISC-V computation generated by trusted transpilation and run on the RISC-V component of the Jolt state. This monadic computation returns a value `v` and an updated RISC-V state `s`. The expression `((JoltISA.execProgram (JoltISA.sllProgramAuto rd rs1 rs2)).run js)` executes the Jolt program on the complete initial state and returns a value `v'` and an updated Jolt state. `System.systemProjectResult` projects from that final Jolt state to a value `v'` and RISC-V state `s'`. Thus, the proof must establish `v = v'` and `s = s'`.

The remaining question is the role of `_h: BinarySourceReadWithLinkedCSRs`. It is an **assumption bundle**, a conjunction of the assumptions needed to close the particular equivalence theorem. We define and justify the core assumptions in Section 4.3; each is supported by the Jolt implementation or by the trusted RISC-V model. For `SLL`, the bundle `BinarySourceReadWithLinkedCSRs rs2 rs1 js` requires that both source registers can be read without changing `js.sail`, and that the six Sail CSR entries—`mstatus`, `mtvec`, `mscratch`, `mepc`, `mcause` and `mtval`—agree with their corresponding Jolt virtual registers in the initial state. These correspondence conditions ensure that `system Project js = js.sail`.

4.1 Advice Completeness and Soundness

The eight division and remainder expansions take an explicit 64-bit advice parameter. In the Lean model, `VirtualAdvice` writes the supplied value to its destination register; subsequent instructions check it. The correctness theorems quantify over this parameter and establish two statements under the instruction's assumption bundle.

Completeness states that the instruction-specific honest advice makes the expansion retire successfully and gives the same projected execution result as Sail. For example, `divw`

`ProgramCompletenessStatement` assumes that `quotient` equals `divw_advice_value h.rs1_val h.rs2_val` and concludes equality with the Sail `DIVW` execution. This honest advice is an internal value used by the expansion; its final word result is obtained by sign extension.

Soundness concerns an arbitrary supplied advice value. For a nonzero destination, any execution returning `.ok RETIRE_SUCCESS js'` has the correct architectural result. In `DIV`, `DIVU` and `DIVW`, the soundness statement establishes that accepted advice equals the honest value; completeness then gives agreement with Sail. For `DIVUW` and the four remainder instructions, the soundness statement directly establishes the correct resulting RISC-V state. Thus the guarantee covers arbitrary accepted advice without requiring every internal advice value to be uniquely determined.

If an advice-checking assertion fails, the Lean execution returns `.error (Error.Assertion ...) js'`, retaining the state at failure and stopping the expansion. Such an execution is excluded by the successful-retirement premise of soundness. For `rd = x0`, all eight expansions instead use a no-op branch that ignores the advice. The architectural result is discarded, so correctness in this branch is independent of the supplied advice; there is no requirement to reject an unused value.

Before describing the assumptions and justifying their presence, we describe the two changes made to the trusted transpiled code.

4.2 Changes to Trusted Transpilation

The RISC-V component is generated from the official Sail specification [14] using the LeanRV64D transpilation [6]. We treat this generated model as trusted, except for the two changes below, which align its behaviour with the Jolt CPU.

```
-- WARNING: CHANGE IN TRANSPILED CODE
def plat_enable_misaligned_access : Bool := false
```

We configure the reference model to reject misaligned memory accesses. The generated configuration originally enables them; disabling them aligns its fault behaviour with the Jolt expansions. The Ethereum zkVM Standards[7] recommend that all zkVMs use `RV64IM+Zicclsm` as the target baseline, where `Zicclsm` is included to support misaligned loads and stores. The model studied here therefore differs from this recommended baseline.

```
-- WARNING: (change to sail, Jolt does not support Zicfilp extension).
| Ext_Zicfilp => false
-- WARNING: (change to sail, Jolt does not support Zicfiss extension).
| Ext_Zicfiss => false
```

Jolt does not implement the Zicfiss and Zicfilp extensions, whereas the trusted transpilation enables them by default. We therefore disable both extensions in the generated model. These changes are the *only* changes we make to the trusted transpiled Lean code.

[7] `https://zkevm.ethereum.foundation/blog/zkevm-standards-v0-release`

4.3 Jolt Assumptions

For the remainder of this section, we list the assumptions used in the Jolt equivalence proofs and justify each one with evidence from the Jolt implementation. The primitive assumptions below are combined in the instruction-specific bundles.

Privilege Is Always Machine Based on the specification, Jolt runs its CPU in machine mode throughout execution[8]. The following hypothesis states that `cur_privilege` is `Privilege.Machine`.

```
structure CurPrivilegeMachine (s : SailState) : Prop where
  value : s.regs.get? Register.cur_privilege =
    some (Privilege.Machine : RegisterType Register.cur_privilege)
```

MPP Field Set to Machine Jolt uses virtual register 39 to model the `mstatus` control status register. The following assumption requires the MPP field of `mstatus` to be `Machine` in the initial state. Jolt's `MRET` implementation[9] leaves this field unchanged; it does not establish the condition.

```
structure MstatusMppMachine (js : SailJoltState) : Prop where
  value_eq :
    _get_Mstatus_MPP (js.vregs JoltISA.mstatusVReg) =
      privLevel_to_bits Privilege.Machine
```

MISA User Mode Enabled Jolt initialises `misa` to `0x8000000008014312f` in its CPU emulator[10]. The `_get_Misa_U` accessor reads bit 20, which is set to one based on the above value. According to the RISC-V specification [14], this indicates that User mode is implemented.

```
-- The Misa register is populated in the hash-map and bit 20 is set to 1
structure MisaUserEnabled (s : SailState) : Prop where
  exists_value : ∃ misa : BitVec 64,
    s.regs.get? Register.misa = some (misa : RegisterType Register.misa)
  ∧
    _get_Misa_U misa = 1#1
```

MPRV Bit The memory equivalence theorems assume that `mstatus.MPRV` is zero. Together with Machine privilege, this makes address translation the identity for the data accesses considered here.[11]

[8] `https://github.com/abiswas3/jolt/blob/main/tracer/src/emulator/cpu.rs#L329-L353`

[9] `https://github.com/abiswas3/jolt/blob/main/tracer/src/instruction/mret.rs#L7-L18`

[10] `https://github.com/abiswas3/jolt/blob/e012da54c3bb26a6436b5ca74e86c19bb39695ad/tracer/src/emulator/cpu.rs#L546`

[11] See `translateAddr_load_data_of_machine_mprv_zero` and the corresponding store and atomic lemmas in `JoltBytecode/InstructionEquivalence/ProofSupport/BundleLemmas.lean`.

```
-- mstatus is present and its MPRV bit is zero
structure MstatusMprvZero (s : SailState) : Prop where
  value : ∃ mval : RegisterType Register.mstatus,
    s.regs.get? Register.mstatus = some mval ∧
    _get_Mstatus_MPRV mval = 0#1
```

Zicfilp Extension Jolt does not support the Zicfilp extension.

```
structure ZicfilpDisabled (s : SailState) : Prop where
  value : currentlyEnabled extension.Ext_Zicfilp s = .ok false s
```

The next assumptions concern writes to control and status registers. Jolt's CSR expansions store the computed value directly, whereas Sail applies the legalisation function for the target CSR. The equivalence theorems therefore assume that this function leaves the written value unchanged. The conditions below specify the values covered by these theorems.

`mtvec` Writes Use Direct Mode Sail accepts `mtvec` values with mode bits `00` (Direct) or `01` (Vectored) unchanged. Our assumption uses Direct mode. The cited ZeroOS boot path writes `_trap_handler` to `mtvec` using `csrw mtvec, t0`[12], and the trap entry is defined under `.align 2`[13]. This gives a four-byte aligned address with mode bits `00`, satisfying the premise for that write.

```
structure MtvecWriteDirectMode (value : BitVec 64) : Prop where
  mode_eq : _get_Mtvec_Mode value = 0b00#2
```

`mepc` Alignment `MepcReadAligned` requires the stored `mepc` value to be unchanged by Sail's `align_pc`. With compressed instructions enabled, this requires its least significant bit to be zero. The condition is a premise on `mepc`, which can be written separately from the current PC.

```
structure MepcReadAligned (value : BitVec 64) (s : SailState) : Prop
  where
  value_eq : align_pc value s = .ok value s
```

`MepcWriteLegalized` applies to the value being written: the source-register value for `CSRRW`, or the bitwise OR of the old CSR value and the source-register value for `CSRRS`. When `CSRRS` has `rs1 = x0`, it performs no CSR write and its bundle requires no write-side legalisation premise.

[12] https://github.com/LayerZero-Labs/ZeroOS/blob/main/platforms/spike-platform/src/boot.rs#L13-L19

[13] https://github.com/LayerZero-Labs/ZeroOS/blob/main/crates/zeroos-arch-riscv/src/trap.rs#L308-L316

```
structure MepcWriteLegalized (value : BitVec 64) : Prop where
  value_eq : legalize_xepc value = value



structure MstatusWriteLegalized
    (old value : BitVec 64) (s : SailState) : Prop where
  value_eq : legalize_mstatus old value s = .ok value s
```

Jolt's CSRRW expansion writes the source-register value directly to the `mstatus` virtual register[14], while CSRRS writes the bitwise OR of the old value and the source-register value when `rs1` is nonzero[15]. `MstatusWriteLegalized old value s` requires Sail's legalisation to return this computed value without changing the state. The expansion does not enforce this condition; it is an explicit premise of the corresponding equivalence theorem.

Virtual-Register Correspondence Jolt models persistent CSRs using virtual registers. We could write directly to the Sail registers in the Jolt semantics, but doing so would require us to change the Jolt expansions. Instead, the semantics write the virtual registers and this assumption states that the virtual registers are the same as the control status registers in the hash map[16].

```
/-- Sail `mstatus` agrees with Jolt's persistent `mstatus` virtual
  register. -/
structure MstatusVRegMatchesSail (js : SailJoltState) : Prop where
  value_eq :
    js.sail.regs.get? Register.mstatus =
      some (js.vregs JoltISA.mstatusVReg)

/-- Sail `mtvec` agrees with Jolt's persistent trap-handler virtual
  register. -/
structure MtvecVRegMatchesSail (js : SailJoltState) : Prop where
  value_eq :
    js.sail.regs.get? Register.mtvec =
      some (js.vregs JoltISA.trapHandlerVReg)

-- 4 more similar assumptions about Mscratch, Mepc, Mcause, Mtval
```

[14] https://github.com/abiswas3/jolt/blob/main/crates/jolt-program/src/expand/control_flow/csrrw.rs#L17-L63

[15] https://github.com/abiswas3/jolt/blob/main/crates/jolt-program/src/expand/control_flow/csrrs.rs#L26-L74

[16] https://github.com/abiswas3/jolt/tree/main/crates/jolt-program/src/expand/allocator.rs

```
structure DwordPresent (addr : BitVec 64) (s : SailState) : Prop where
  bytes :
    ∃ bytes : Fin 8 → BitVec 8,
      ∀ k : Fin 8, s.mem.get? (addr.toNat + k.val) = some (bytes k)
```

Jolt implements narrow loads by reading the enclosing eight-byte aligned doubleword and extracting the requested bytes. In the load assumption bundle, `DwordPresent` is applied at the base of this aligned window and requires all eight bytes to be present in the initial Sail memory map. Together with the access-permission and ordinary-RAM assumptions below, this supplies the readable window used by the expansion.

```
abbrev LoadPmpOk (addr : BitVec 64) (width : Nat) (s : SailState) : Prop
  :=
  phys_access_check (Load Data) Privilege.Machine
    (physaddr.Physaddr addr) width false s = .ok none s

structure LoadPmpOkWindow
    (base : BitVec 64) (width : Nat) (s : SailState) : Prop where
  ok :
    ∀ offset accessWidth : Nat, offset + accessWidth ≤ width →
      LoadPmpOk (base + BitVec.ofNat 64 offset) accessWidth s

abbrev StorePmpOk (addr : BitVec 64) (width : Nat) (s : SailState) : Prop
   :=
  phys_access_check (Store Data) Privilege.Machine
    (physaddr.Physaddr addr) width false s = .ok none s

structure StorePmpOkWindow
    (base : BitVec 64) (width : Nat) (s : SailState) : Prop where
  ok :
    ∀ offset accessWidth : Nat, offset + accessWidth ≤ width →
      StorePmpOk (base + BitVec.ofNat 64 offset) accessWidth s

abbrev AtomicPmpOk
    (op : amoop) (addr : BitVec 64) (width : Nat) (s : SailState) : Prop
  :=
  phys_access_check (Atomic (op, Data, Data)) Privilege.Machine
    (physaddr.Physaddr addr) width true s = .ok none s

structure AtomicPmpOkWindow
    (op : amoop) (base : BitVec 64) (width : Nat) (s : SailState) : Prop
  where
  ok :
    ∀ offset accessWidth : Nat, offset + accessWidth ≤ width →
      AtomicPmpOk op (base + BitVec.ofNat 64 offset) accessWidth s
```

Jolt's Rust MMU explicitly does not implement memory protection[17]. These assumptions require Sail's physical-access checks to accept the relevant reads, writes or

[17] https://github.com/abiswas3/jolt/blob/main/tracer/src/emulator/mmu.rs#L17

atomic accesses. The window forms require acceptance for every access contained in the eight-byte window, covering both Jolt's doubleword access and Sail's narrower access.

The generated RISC-V model distinguishes ordinary RAM from memory-mapped I/O. The memory theorems presented here use the ordinary-memory model and require the relevant window to avoid Sail's readable or writable MMIO regions. Jolt has a separate device-I/O path; these assumptions delimit the memory behaviour covered by the theorems.

```
abbrev NotReadableMmio (addr : BitVec 64) (width : Nat) (s : SailState) :
   Prop :=
  within_mmio_readable (physaddr.Physaddr addr) width s = .ok false s

structure NotReadableMmioWindow
    (base : BitVec 64) (width : Nat) (s : SailState) : Prop where
  ok :
    ∀ offset accessWidth : Nat, offset + accessWidth ≤ width →
      NotReadableMmio (base + BitVec.ofNat 64 offset) accessWidth s

abbrev NotWritableMmio (addr : BitVec 64) (width : Nat) (s : SailState) :
   Prop :=
  within_mmio_writable (physaddr.Physaddr addr) width s = .ok false s

structure NotWritableMmioWindow
    (base : BitVec 64) (width : Nat) (s : SailState) : Prop where
  ok :
    ∀ offset accessWidth : Nat, offset + accessWidth ≤ width →
      NotWritableMmio (base + BitVec.ofNat 64 offset) accessWidth s




structure XRegReadable (r : regidx) (s : SailState) : Prop where
  exists_value : ∃ value : BitVec 64, rX_bits r s = .ok value s

structure SailRegReadable (r : Register) (s : SailState) : Prop where
  exists_value : ∃ value : RegisterType r, s.regs.get? r = some value

/-- Assumptions for an instruction that reads one architectural source
  register. -/
structure UnarySourceReadAssumptions (rs1 : regidx) (s : SailState) where
  rs1_val : BitVec 64
  rs1_read : rX_bits rs1 s = .ok rs1_val s

/-- Assumptions for an instruction that reads two architectural source
  registers. -/
structure BinarySourceReadAssumptions
    (rs2 rs1 : regidx) (s : SailState) where
  rs1_val : BitVec 64
  rs1_read : rX_bits rs1 s = .ok rs1_val s
  rs2_val : BitVec 64
```

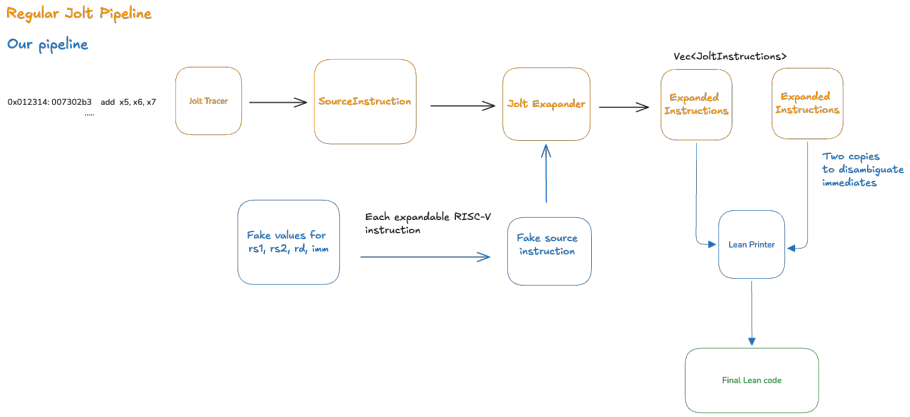


Fig. 3. An outline of how we reuse the Jolt expansion pipeline to obtain Lean translations.

```
rs2_read : rX_bits rs2 s = .ok rs2_val s
```

Much like memory, these structures state that the RISC-V registers can be read successfully. `UnarySourceReadAssumptions` applies to instructions that read one source register; `BinarySourceReadAssumptions` requires both source registers to be readable.

5 Auto-extraction

Figure 3 summarises the extraction process. The `jolt-lean-gen` tool invokes Jolt's production Rust expander on concrete source instructions and renders its output as parameterised Lean `Program` definitions.[18] Reusing the production expander avoids duplicating its expansion rules. The rendering and generalisation from concrete operands to Lean parameters remain trusted parts of the extraction.

The production entry point `expand_instruction` accepts a `SourceInstruction`, an `ExpansionAllocator` and a target instruction profile. It constructs the expansion, allocates virtual registers and recursively expands intermediate source instructions to obtain final Jolt instructions. The generator calls this entry point with a fresh allocator and the `RV64IMAC_JOLT` profile, then converts each returned instruction into a `Jolt InstructionRow`. The relevant data structures are:

```
pub struct JoltInstructionRow {
    pub instruction_kind: JoltInstructionKind,
    pub address: usize,
    pub operands: NormalizedOperands,
    pub virtual_sequence_remaining: Option<u16>,
```

[18] The generator is implemented in `crates/jolt-lean-gen/src/main.rs`; this description refers to Rust commit `d39bd518a65e`.

```
    pub is_first_in_sequence: bool,
    pub is_compressed: bool,
}

pub struct NormalizedOperands {
    pub rs1: Option<u8>,
    pub rs2: Option<u8>,
    pub rd: Option<u8>,
    pub imm: i128,
}
```

The printer uses `instruction_kind` to select a Lean constructor and `operands` to supply its arguments. Register indices below 32 denote architectural registers; larger indices denote virtual registers. Row addresses and sequence metadata are checked between the sampled expansions but are not stored in the sequential Lean `Program`. The printer also supplies model-specific arguments: the source instruction determines the load fault class and alignment exception, and advice rows receive explicit `BitVec 64` parameters named `advice0`, `advice1`, and so on.

5.1 Recovering Parameters

The generator uses `rd = x1`, `rs1 = x2` and `rs2 = x3` where those operands occur, and replaces these register indices with the corresponding Lean parameters when printing. For instructions with a destination register, it separately expands `rd = x0` and emits an `if isX0 rd then ... else ...` definition. This captures the production pipeline's zero-destination handling: pure computations can become a no-op, while instructions with side effects can use a virtual destination.

Immediate operands need additional handling because the rows store concrete values without recording how they were computed. For each destination arm, the generator compares expansions at source immediates 37 and 41, or 5 and 13 for the five-bit word shifts. It rejects changes in sequence length or any non-immediate row field, and rejects a row whose immediate equals the first sample but does not track the second. For loads and stores, a row immediate equal to the respective source immediate in both runs is printed as `imm`. Shift-immediate instructions instead use explicit symbolic rules for the required multipliers and bit masks, parameterised by a five- or six-bit `shamt`.

These checks detect certain inconsistencies between the two runs. Generalising the sampled register assignments and immediates still assumes that the resulting templates describe every operand choice covered by the Lean theorem, including register aliasing and other immediate values. This property is not established by the sample checks.

5.2 Example and Extraction Scope

For example, with `rd = x1`, `rs1 = x2` and `imm = 37`, the `LB` expansion has the following rows:

```
VirtualAlignAddr   v41, x2, 37
LD                 v41, v41, 0
VirtualWindowMaskB v40, x2, 37
VirtualPextSigned  x1, v41, v40
```

The corresponding nonzero-destination arm of `lbProgramAuto` is shown below, using the instruction interface of the paper's Lean model.

```
.instr (.VirtualAlignAddr
  (.vreg (BitVec.ofNat 7 41)) (.xreg rs1) imm) <|
.instr (.LD .normal
  (.vreg (BitVec.ofNat 7 41)) (.vreg (BitVec.ofNat 7 41))
  (0 : BitVec 12)) <|
.instr (.VirtualWindowMaskB
  (.vreg (BitVec.ofNat 7 40)) (.xreg rs1) imm) <|
.instr (.VirtualPextSigned (.xreg rd)
  (.vreg (BitVec.ofNat 7 41)) (.vreg (BitVec.ofNat 7 40))) <|
.done RETIRE_SUCCESS
```

Automatic extraction omits `ECALL`, `EBREAK`, `MRET`, `CSRRW`, `CSRRS`, `LR.W`, `LR.D`, `SC.W`, `SC.D` and registered `Inline` expansions. System and reservation expansions in the Lean model are maintained separately. In particular, the covered CSR expansions are hand-written; being excluded from the generator does not itself mean an instruction is excluded from verification. The theorem coverage is listed in Section 6.

Lean type checking establishes that the resulting definitions are well typed, and the instruction proofs establish their stated behaviour in the Lean model. Applying those results to Rust additionally assumes that extraction preserves the instruction sequence, operand mapping and immediate interpretation for every covered input. A well-typed rendering error or an unrepresented operand-dependent expansion branch could violate that assumption. Reusing the production expander limits duplicated logic, but does not remove this correspondence obligation.

6 Results

Our Lean inventory contains 91 RISC-V instruction kinds: 33 native kinds and 58 expanded kinds.[19] We establish correctness results for all 33 native kinds and 51 expanded kinds, leaving seven expanded kinds excluded. We count each instruction constructor once: register choices, immediate values, advice values and atomic ordering flags do not create additional kinds. `CSRRW` and `CSRRS` each count once, with coverage restricted to the six CSRs described below. Virtual instructions and advice-load operations are not guest RISC-V instruction kinds and are not included in these counts.

In the process, we found an incorrect expansion that prevented trace generation for a valid execution. We also identified a mismatch between Jolt's `MRET` expansion and the Sail semantics under the stated assumptions. Below we describe our findings in detail.

[19] The paper uses the instruction interface and inventory at Jolt-QED commit `b3f60950aeb7` (initial release). The inventory is `RiscvInstruction` in `JoltBytecode/RiscvInstruction.lean`. Counts refer to completed instruction-specific results, excluding unfinished cases in the aggregate dispatcher.

Table 1. Instruction coverage under the corresponding assumption bundles. The covered expanded rows total 51; the native row contributes a further 33. Atomic results concern execution in the sequential model.

Category	Instruction kinds	Count
Native	ADD, ADDI, ADDIW, ADDW, AND, ANDI, ANDN, AUIPC, BEQ, BGE, BGEU, BLT, BLTU, BNE, FENCE, JAL, JALR, LD, LUI, MUL, MULHU, MULW, OR, ORI, SD, SLT, SLTI, SLTIU, SLTU, SUB, SUBW, XOR, XORI	33
Expanded shifts	SLL, SLLI, SLLW, SLLIW, SRL, SRLI, SRLW, SRLIW, SRA, SRAI, SRAW, SRAIW	12
Expanded multiplication	MULH, MULHSU	2
Expanded division / remainder	DIV, DIVU, DIVW, DIVUW, REM, REMU, REMW, REMUW	8
Expanded loads	LB, LBU, LH, LHU, LW, LWU	6
Expanded stores	SB, SH, SW	3
Expanded atomics	AMOSWAP, AMOADD, AMOXOR, AMOAND, AMOOR, AMOMIN, AMOMAX, AMOMINU, AMOMAXU, each in .W and .D forms	18
Expanded CSR operations	CSRRW, CSRRS	2
Excluded expansions	ECALL, EBREAK, LR.W, LR.D, SC.W, SC.D, MRET	7

6.1 Incompleteness Issue

We identified a bug in the original implementation of Jolt's DIVW sequence. In this historical expansion, the tracer supplied a signed quotient and an unsigned remainder magnitude as untrusted advice. The magnitude was an auxiliary value used to check the division; DIVW writes only the quotient to the architectural destination register. Completeness, as described in Section 4.1, requires honest advice to produce a correct, normally retiring execution under the instruction's assumptions.

For rs1 = i32::MIN and rs2 = 0, the tracer supplied quotient -1 and remainder magnitude 2^{31}. The expansion applied SRAI with shift amount 31 to the magnitude, then used VirtualAssertEQ to require that the result equal zero. For 2^{31}, the shift produces 1, so this assertion panicked during trace generation. A valid RISC-V execution therefore failed to produce a completed trace despite honest advice, demonstrating an incompleteness bug.

The fix changed this shift amount from 31 to 32, allowing the magnitude to occupy the full unsigned 32-bit range. It was merged in pull request #1473 on 29 April 2026.[20] The current Lean artefact proves completeness and soundness for a later DIVW expansion,

[20] Jolt PR #1473, merged as commit e26d423576d2.

which takes quotient advice and computes the remainder magnitude internally. The two-advice sequence above describes the historical bug.

6.2 Coverage Limitations

The seven exclusions have three distinct causes: opaque reservation operations for `LR.W`, `LR.D`, `SC.W` and `SC.D`; differing treatment of traps for `ECALL` and `EBREAK`; and the `MRET` state mismatch described below.

Load-Reserved and Store-Conditional The reference model represents reservation operations using axioms without operational definitions. These opaque operations prevent the current proofs from establishing equivalence for `LR.W`, `LR.D`, `SC.W` and `SC.D`. The declarations are:

```
axiom load_reservation : Arch.pa → Nat → SailM Unit
axiom match_reservation : Arch.pa → Bool
axiom cancel_reservation : Unit → SailM Unit
axiom valid_reservation : Unit → Bool
```

ECALL and EBREAK The Sail-derived Lean model defines both instructions, returning a `Trap` result at the instruction-execution boundary. Jolt's `ECALL` expansion instead writes virtual trap CSRs and transfers control to the handler, while its `EBREAK` expansion emits a self-loop used as a termination marker. The corresponding Lean Jolt programs return `RETIRE_SUCCESS`. The present equivalence results do not reconcile these different treatments of traps, so both instructions are excluded.

Control and Status Registers (CSRs) The verified `CSRRW` and `CSRRS` cases cover six CSRs: `mscratch`, `mcause`, `mtval`, `mtvec`, `mepc` and `mstatus`. Sail accepts writes to `mscratch`, `mcause` and `mtval` unchanged. For `mtvec`, `mepc` and `mstatus`, the equivalence theorems use the legalisation premises described in Section 4, applied to the value computed by the instruction. These are conditions on the covered writes; Jolt's expansions do not legalise arbitrary CSR values.

MRET With `mstatus.MPP` initially set to Machine, Sail's `MRET` retains Machine privilege but resets MPP to User because `misa` advertises U-mode support. Jolt's expansion leaves `mstatus` unchanged, so the final states differ. This mismatch prevents us from closing the equivalence theorem. The table below illustrates the main proof bottlenecks.

Table 2. Selected Sail and projected Jolt state components before and after `MRET`, with initial MPP=Machine, U-mode advertised by `misa`, and an aligned `mepc`. Both models update `nextPC`; `PC` is unchanged at this instruction-execution boundary.

Before `MRET`	
Sail	**Jolt**
`PC` = current `pc`	`PC` = current `pc`
`nextPC` = old `nextPC`	`nextPC` = old `nextPC`
`cur_privilege` = Machine	`cur_privilege` = Machine
`mepc` = return target	`mepc` = return target
`mstatus.MPP` = Machine	`mstatus.MPP` = Machine
`mstatus.MIE` = old `MIE`	`mstatus.MIE` = old `MIE`
`mstatus.MPIE` = old `MPIE`	`mstatus.MPIE` = old `MPIE`
`mstatus.MPRV` = old `MPRV`	`mstatus.MPRV` = old `MPRV`
After `MRET`	
Sail	**Jolt**
`nextPC` = `mepc`	`nextPC` = `mepc`
`cur_privilege` = Machine from old `mstatus.MPP`	`cur_privilege` = Machine
`mepc` = unchanged	`mepc` = unchanged
`mstatus.MPP` = User mismatch under the stated assumptions	`mstatus.MPP` = Machine unchanged
`mstatus.MIE` = old `MPIE` mismatch unless old `MIE` = old `MPIE`, but resolvable	`mstatus.MIE` = old `MIE` unchanged
`mstatus.MPIE` = 1 mismatch unless old `MPIE` = 1, but resolvable	`mstatus.MPIE` = old `MPIE` unchanged
`mstatus.MPRV` = old `MPRV` unchanged because privilege remains Machine	`mstatus.MPRV` = old `MPRV` unchanged

7 Future Work And Conclusion

We have established correctness results for 51 expanded and 33 native instruction kinds in the Lean model, with the seven exclusions listed above. The results compose under the conditions stated in Section 4. Applying these results to the Rust implementation relies on the semantic and extraction correspondence described above. The question of how to further reduce this trust surface is an interesting research direction. Our extraction procedure reuses the production expander, while trusting the generalisation and rendering of its output. Establishing this correspondence formally, or investigating Rust-to-Lean translation approaches such as Aeneas [7], could reduce the remaining trust assumptions.

Later stages of Jolt's proving pipeline involve more complex Rust code, requiring further work on extraction and modelling. Bytecode expansion provides a foundation for this effort. Our instruction semantics tell Lean how to execute each Jolt instruction. They therefore provide the basis for modelling a prover that honestly runs a Jolt program and records the resulting witness. Witness and constraint modelling is in progress in the repository's `JoltConstraints` development.

The next phase is to prove completeness and soundness of Jolt's constraints with respect to this execution model. Completeness says that if the prover traces the Jolt program honestly, then the resulting witness values satisfy every constraint. Soundness says that if the prover's witness values satisfy every constraint, then the prover is in possession of an honest witness for the claimed Jolt execution, consistent with its public inputs and outputs. The constraints are imposed directly on the witness values.

Running the tracer is one way for the prover to obtain an honest witness. The soundness claim concerns possession of that witness, however it was obtained.

After establishing this correspondence between execution and constraints, we would verify Jolt's sum-check protocols [17], initially assuming an ideal polynomial-opening oracle. Finally, we would prove that the polynomial commitment scheme securely realises the required oracle functionality. Defining the Jolt ISA and proving its instruction-level agreement with RISC-V are foundational steps towards this broader verification.

References

1. Armstrong, A., Bauereiss, T., Campbell, B., Gray, K., Norton-Wright, R., Mundkur, P., Wassell, M., French, J., Pulte, C., Flur, S., Stark, I., Krishnaswami, N., Sewell, P.: Sail: A Language for ISA Semantics, https://github.com/rems-project/sail, REMS Project, University of Cambridge
2. Arun, A., Setty, S., Thaler, J.: Jolt: SNARKs for Virtual Machines via Lookups. In: Advances in Cryptology – EUROCRYPT 2024 (2024)
3. Beringer, L., Petcher, A., Ye, K.Q., Appel, A.W.: Verified Correctness and Security of OpenSSL HMAC. In: 24th USENIX Security Symposium (USENIX Security 15). pp. 207–221. USENIX Association, Washington, D.C. (Aug 2015), https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/beringer
4. Ethereum Foundation: Allocation Update – Q3 2025 (Dec 2025), https://blog.ethereum.org/en/2025/12/02/allocation-q3-25, Ethereum Foundation Blog
5. Ethereum Foundation: Shipping an L1 zkEVM #2: The Security Foundations (Dec 2025), https://blog.ethereum.org/2025/12/18/zkevm-security-foundations, Ethereum Foundation Blog
6. Grosser, T., Stefanesco, L., Parker, J., Newcomb, L., Selfridge, B., Hamlin, B., von Raumer, J., Lahfa, R.: LeanRV64D: Lean 4 Model of RISC-V RV64GC Generated from the Sail Specification (2024), https://github.com/opencompl/sail-riscv-lean, University of Cambridge, Galois Inc., and LindyLabs. Advisors: Peter Sewell, Alasdair Armstrong, Brian Campbell. Funded by the Ethereum Foundation Verified zkEVM Project
7. Ho, S., Protzenko, J.: Aeneas: Rust Verification by Functional Translation. Proceedings of the ACM on Programming Languages **6**(ICFP) (Aug 2022). https://doi.org/10.1145/3547647, https://doi.org/10.1145/3547647
8. Hochrainer, C., Wuestholz, V., Christakis, M.: Arguzz: Testing zkVMs for Soundness and Completeness Bugs. In: USENIX Security Symposium (2026)
9. LayerZero: Zero: Technical Positioning Paper (Feb 2026), https://layerzero.network/blog/zero-technical-positioning-paper, Accessed 2026-07-27
10. Nair, V., Thaler, J., Zhu, M.: Proving CPU Executions in Small Space. Cryptology ePrint Archive, Paper 2025/611 (2025), https://eprint.iacr.org/2025/611, Appendix A
11. Nethermind Security Formal Verification Team: Proof-of-Concept Formal Verification of SP1 zk Chips (May 2025), https://www.nethermind.io/blog/proof-of-concept-formal-verification-of-sp1-zk-chips, Project report
12. Nethermind Security Formal Verification Team: Towards Formal Verification of the First RISC-V zkVM (Feb 2025), https://www.nethermind.io/blog/towards-formal-verification-of-the-first-risc-v-zkvm, Project report
13. RISC-V Foundation: The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, document version 20191213 edn. (Dec 2019), https://docs.riscv.org/reference/isa/v20191213/_attachments/riscv-unprivileged.pdf
14. RISC-V International, REMS Project, University of Cambridge: The RISC-V Sail Model: Official Formal Specification of the RISC-V Architecture, https://github.com/riscv/sail-riscv, The official machine-readable specification of the RISC-V ISA, adopted by RISC-V International
15. Setty, S., Thaler, J.: Twist and Shout: Faster memory checking arguments via one-hot addressing and increments. Cryptology ePrint Archive, Paper 2025/105 (2025), https://eprint.iacr.org/2025/105
16. Succinct Labs: Formal Verification of SP1 Hypercube (Oct 2025), https://blog.succinct.xyz/nethermind-lean/, Project announcement

17. Thaler, J.: Sum-check Is All You Need: An Opinionated Survey on Fast Provers in SNARK Design. Cryptology ePrint Archive, Paper 2025/2041 (2025), https://eprint.iacr.org/2025/2041
18. The mathlib Community: The Lean Mathematical Library. In: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs. pp. 367–378 (2020). https://doi.org/10.1145/3372885.3373824
19. Tuma, D., Hopper, N.: VCVio: A Formally Verified Forking Lemma and Fiat–Shamir Transform, via a Flexible and Expressive Oracle Representation. Cryptology ePrint Archive, Paper 2024/1819 (2024), https://eprint.iacr.org/2024/1819
20. Verified zkEVM: ArkLib: Formally Verified Arguments of Knowledge in Lean (2025), https://github.com/Verified-zkEVM/ArkLib, Software repository
21. Wen, H., Stephens, J., Chen, Y., Ferles, K., Pailoor, S., Charbonnet, K., Dillig, I., Feng, Y.: Practical Security Analysis of Zero-Knowledge Proof Circuits. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 1471–1487. USENIX Association (2024), https://www.usenix.org/conference/usenixsecurity24/presentation/wen
22. Yang, X., Chen, Y., Eide, E., Regehr, J.: Finding and Understanding Bugs in C Compilers. In: Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation. pp. 283–294 (2011). https://doi.org/10.1145/1993316.1993532